

CAPSTONE PROJECT · PART I

CP353301 Internetworking

Introduction to Automation Tools

Embedded Event Manager

Agent-Based

Agentless

ทฤษฎีเชิงลึก · สาธิต CLI จริง · Troubleshooting · แนวโน้มอนาคต

กลุ่ม 9 — หัวข้อที่ 9: Automation

อ้างอิง CCNP Enterprise: Core Networking, Chapter 29

มหาวิทยาลัยขอนแก่น · 17 กันยายน 2569

สมาชิกกลุ่ม

01

นายเกรียงไกร ประเสริฐ

รหัสนักศึกษา 663380616-4

02

นายปิติ มุลเทพพิชัย

รหัสนักศึกษา 663380045-1

03

นางสาวภัทราวดี ส่องศรีโรจน์

รหัสนักศึกษา 663380610-6

04

นายจีรภัทร แก้วดี

รหัสนักศึกษา 673380577-9

โครงสร้างการนำเสนอ 5 ส่วน

เวลานำเสนอประมาณ 18 นาที (รวมการสาธิต) และช่วงถาม-ตอบ 5 นาที

1

15% · ~3 นาที

แนวคิดและสถาปัตยกรรม

- Problem & Purpose
- EEM / Agent-based / Agentless
- กลไกภายในของแต่ละแบบ

2

10% · ~2 นาที

สถานะปัจจุบันและมาตรฐาน

- Timeline และ RFC ที่เกี่ยวข้อง
- เวอร์ชันล่าสุดปี 2026
- สิ่งที่เปลี่ยนไปจากตำรา

3

40% · ~7 นาที

สาธิต CLI และ Troubleshooting

- Lab บน Cisco CML
- Ansible + EEM แบบ live
- จำลองปัญหา 2 กรณี

4

15% · ~3 นาที

การเปรียบเทียบ

- เทียบ 6 เครื่องมือ
- Trade-off matrix 5 มิติ
- ควรเลือกใช้เมื่อไร

5

10% · ~2 นาที

ทิศทางในอนาคต

- Event / Model-driven, AI
- GitOps และ Zero Trust
- Industry takeaway

เปิด

1

2

3 · Demo & Troubleshooting 40%

4

5

สรุป

สัดส่วนเวลาตามคำแนะนำการนำเสนอ Capstone Project (ส่วนที่ 3 เป็นส่วนหลัก)

Problem & Purpose: ทำไมต้อง Automate

การตั้งค่าทีละเครื่องผ่าน CLI (box-by-box) ไม่ทันกับขนาดและความเร็วของเครือข่ายปัจจุบัน



งานซ้ำ ๆ บนอุปกรณ์จำนวนมาก

hostname, IP, ACL, AAA, SNMP, NTP ต้องพิมพ์ซ้ำทีละเครื่อง ยิ่งเครือข่ายโตยิ่งใช้เวลา



Human error และ Configuration drift

พิมพ์ผิดหรือลืมบางเครื่อง ทำให้ config แต่ละเครื่องค่อย ๆ ไม่ตรงกับมาตรฐาน



ตอบสนองต่อเหตุขัดข้องช้า

ต้องรอคนเห็น alarm แล้ว login เข้าไปเก็บข้อมูลเอง หลักฐานบางส่วนหายไปก่อน



ตรวจสอบย้อนหลัง (Audit) ได้ยาก

ไม่มีบันทึกชัดเจนว่าใครเปลี่ยนอะไร เมื่อไร และเพราะอะไร

<10% → 30%

สัดส่วนองค์กรที่ automate งานเครือข่ายเกินครึ่ง (กลางปี 2023 → คาดการณ์ปี 2026)

ที่มา: Gartner, 18 ก.ย. 2024

เป้าหมายของ Automation Tools

- ✓ **เร็วขึ้น** เปลี่ยน config หลายร้อยเครื่องในครั้งเดียว
- ✓ **สม่ำเสมอ** บังคับ desired state และตรวจจับ drift
- ✓ **ตอบสนองทันที** event → action โดยอัตโนมัติ
- ✓ **ตรวจสอบได้** config เป็นโค้ด เก็บเวอร์ชันใน Git

สถาปัตยกรรม: 3 แนวทางของ Automation

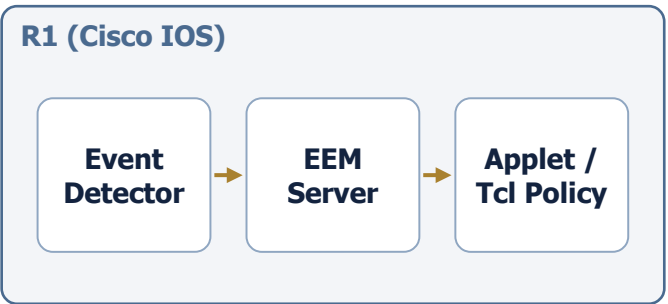
แบ่งตาม "ตรรกะทำงานที่ไหน" และ "อุปกรณ์ปลายทางต้องติดตั้งซอฟต์แวร์เพิ่มหรือไม่"



On-box

Embedded Event Manager (EEM)

R1 (Cisco IOS)



ทำงาน: ภายใน Cisco IOS / IOS XE ไม่ต้องมี server

รูปแบบ: Event-driven (if event → then action)

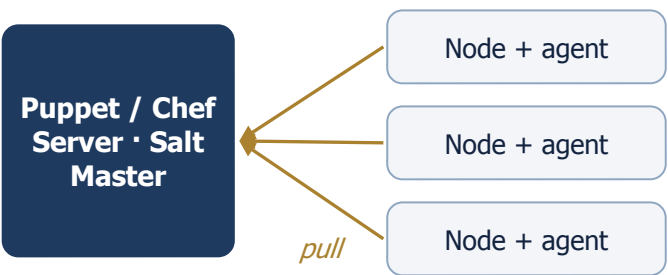
ภาษา: CLI applet หรือ Tcl script

Transport: ไม่มี (ประมวลผลในเครื่อง)



Off-box · Agent-based

Puppet · Chef · SaltStack



ทำงาน: ต้องติดตั้ง agent / minion บนปลายทาง

รูปแบบ: Pull ตามรอบเวลา (Salt: push + event bus)

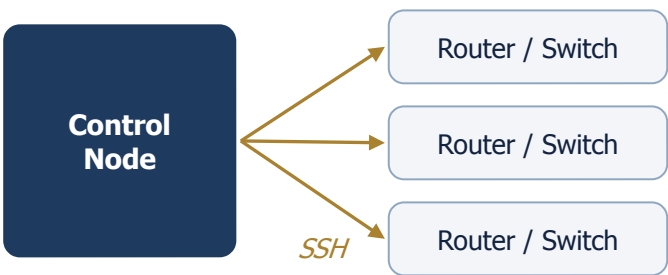
Port: Puppet 8140 · Chef 443 · Salt 4505/4506

เหมาะกับ: server / OS จำนวนมาก



Off-box · Agentless

Ansible · Puppet Bolt · Salt SSH



ทำงาน: ไม่ต้องติดตั้งอะไรบนอุปกรณ์

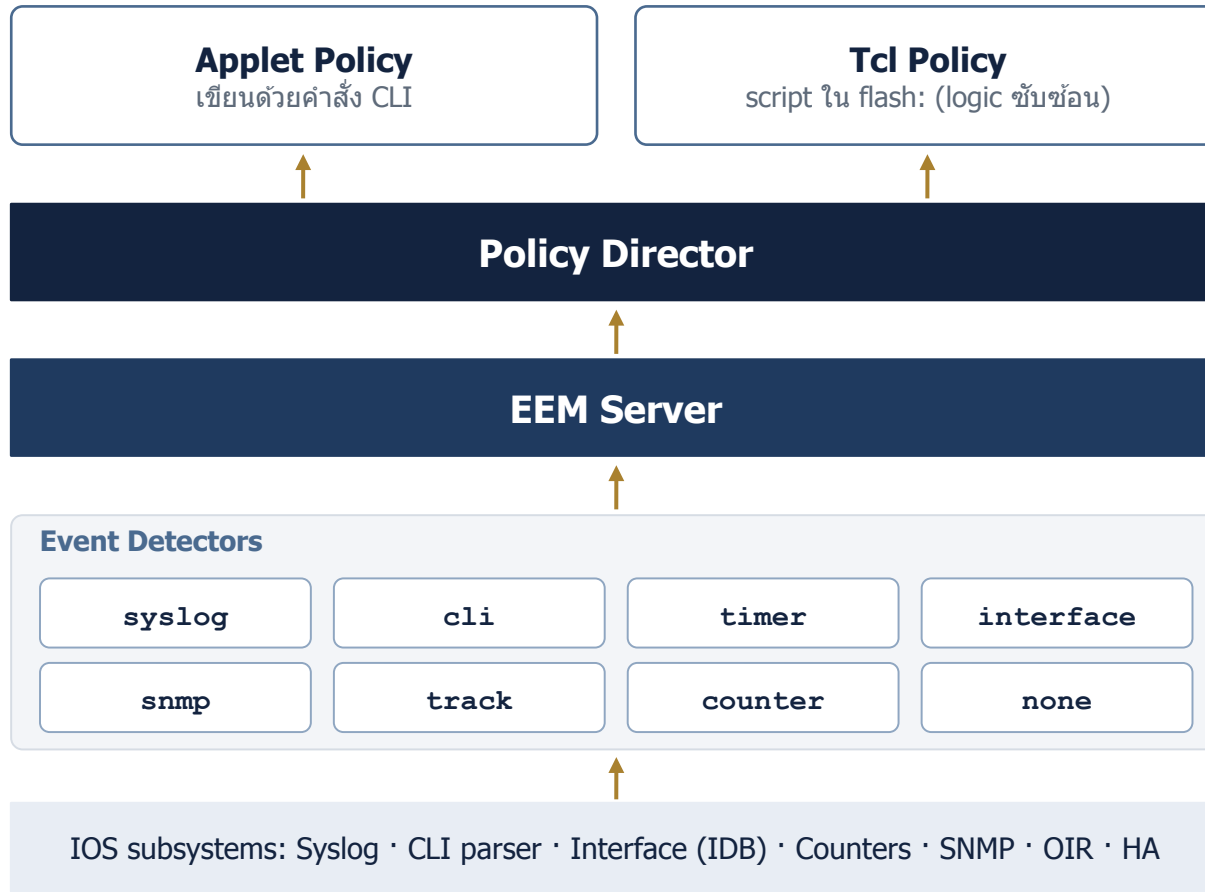
รูปแบบ: Push จาก control node

Transport: SSH 22 · WinRM 5985/5986 · HTTPS API

เหมาะกับ: router / switch ที่ลง agent ไม่ได้

EEM: กลไกการทำงานภายในอุปกรณ์

Event Detector ตรวจจับเหตุการณ์ → EEM Server จับคู่กับ policy ที่ลงทะเบียนไว้ → Policy Director สั่งรัน action



โครงสร้าง applet (ปรับจาก Example 29-1)

```

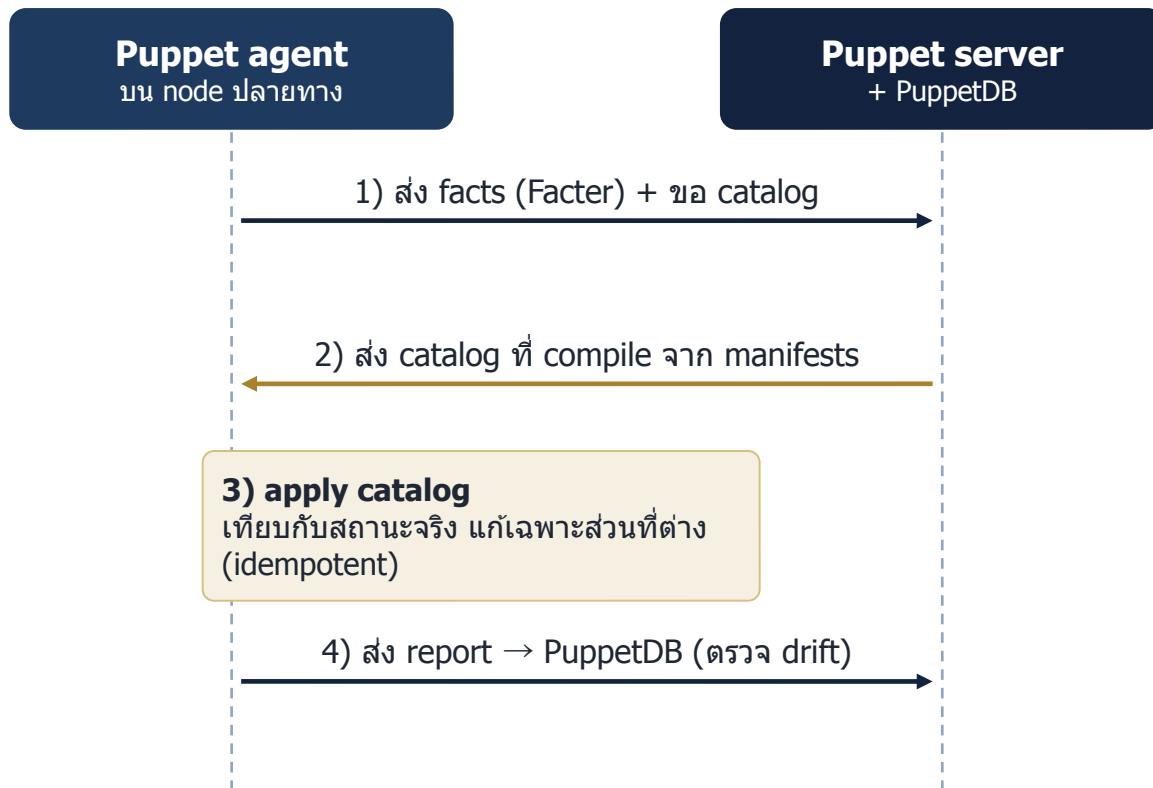
R1(config)# event manager applet LOOP0
event syslog pattern "Interface Loopback0.* down"
action 1.0 cli command "enable"
action 2.0 cli command "config terminal"
action 3.0 cli command "interface loopback0"
action 4.0 cli command "no shutdown"
action 5.0 syslog msg "Loopback0 recovered by EEM"
    
```

event	เงื่อนไขที่ทำให้ applet ทำงาน (EEM 3.0+ รวมหลาย event ได้ด้วย tag + trigger)
action <label>	เรียงตามตัวอักษร (lexicographic) → "10.0" มาก่อน "2.0" จึงควรใช้เลขหลักเท่ากัน
cli "enable"	session ของ applet เริ่มที่ user EXEC ต้องยกกระดับก่อนส่งคำสั่งสิทธิ์สูง
maxrun	applet ถูกยกเลิกเมื่อรันเกิน 20 วินาที (ค่าเริ่มต้น) ปรับได้ด้วย maxrun

ปรับจาก Figure 29-1 (EEM Event Detectors), CCNP Enterprise Core Ch.29

Agent-Based: รอบการทำงานแบบ Pull

agent บนปลายทางรายงานสถานะ แล้วดึง desired state มาบังคับใช้เอง และทำซ้ำตามรอบเวลา (ตัวอย่าง: Puppet)



ศัพท์เทียบกันของ 3 เครื่องมือ

บทบาท	Puppet	Chef	SaltStack
Server	Puppet server	Chef Infra Server	Salt master
Client	Puppet agent	chef-client	salt-minion
โค้ด config	Manifest (.pp)	Recipe (.rb)	State (.sls)
ชุดโค้ด	Module	Cookbook	Formula
ข้อมูล node	Facts (Facter)	Ohai	Grains
ข้อมูล/ความลับ	Hiera	Data bags	Pillar
ภาษา	Puppet DSL	Ruby DSL	YAML + Jinja2
Port	8140	443	4505 / 4506

SaltStack ต่างออกไป: master ส่งงานผ่าน ZeroMQ event bus — beacon บน minion แจ้งการเปลี่ยนแปลง แล้ว reactor บน master สั่งแก้ไขทันที

ทำซ้ำทุก **runinterval** (ค่าเริ่มต้น 30 นาที) · HTTPS 8140 + mutual TLS certificate

Agentless: Ansible คุยกับ Router อย่างไร

โมดูลเครือข่ายรันบน control node แล้วส่งคำสั่ง CLI ผ่าน SSH (network_cli) — อุปกรณ์ไม่ต้องมี Python หรือ agent



Idempotency: หัวใจของ desired state

- 1 อ่าน running-config ปัจจุบันของอุปกรณ์
- 2 เปรียบเทียบกับ desired state ใน playbook
- 3 ส่งเฉพาะคำสั่งที่ขาด → ผลเป็น changed
- 4 ถ้าตรงอยู่แล้ว ไม่ส่งอะไรเลย → ผลเป็น ok

องค์ประกอบของ Playbook (Table 29-5)

Playbook — ชุดของ play สำหรับระบบปลายทาง

Play — ชุดของ task ที่ใช้กับ host หรือกลุ่ม host

Task — การเรียก module 1 ครั้ง

Bolt / Salt SSH ใช้แนวคิด agentless แบบเดียวกัน (Salt SSH ต้องมี Python บนปลายทาง)

Timeline, มาตรฐาน และเวอร์ชันล่าสุด

Automation Tool ไม่มี RFC ของตัวเอง แต่ทำงานบนมาตรฐานเปิดของ IETF — ข้อมูล ณ กันยายน 2026



มาตรฐานที่เครื่องมือเหล่านี้ใช้งาน

มาตรฐาน	เอกสาร	บทบาทใน Automation
SSHv2	RFC 4251–4254	transport ของ Ansible / Bolt / Salt SSH
Syslog	RFC 5424	แหล่ง event ของ EEM และ EDA
NETCONF	RFC 6241 / 6242	config API แบบ transaction ผ่าน SSH
YANG 1.1	RFC 7950	data model ของ config และ state
RESTCONF	RFC 8040	จัดการ YANG data ผ่าน HTTPS
YAML 1.2.2	yaml.org (2021)	ภาษา playbook / Salt state

เวอร์ชันและสถานะล่าสุด

EEM 4.0	Cisco IOS / IOS XE — ตรวจสอบด้วย show event manager version
ansible-core 2.21	ออก พ.ค. 2026 · ใช้คู่กับ collection cisco.ios
Salt 3008 LTS	Broadcom ดูแล · สาย 3007 หมดอายุ ก.ค. 2026
Chef Infra Client 19	รุ่น LTS ของ Progress · ใช้ระบบ license key
Puppet Core / OpenVox	Perforce (เชิงพาณิชย์) / Vox Pupuli (open source)

Key Improvements: ตำรา → ปี 2026

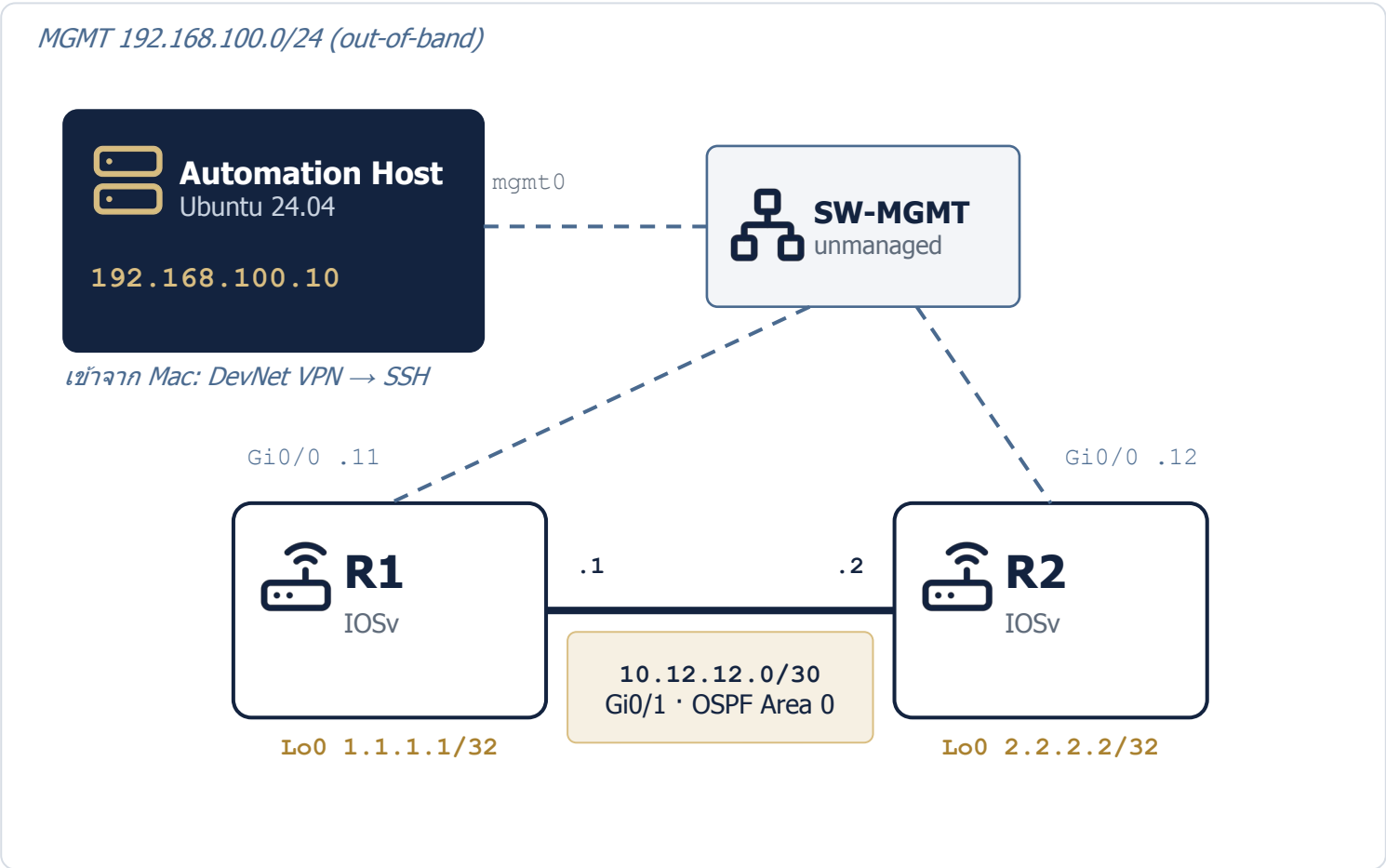
สิ่งที่เปลี่ยนไปเมื่อเทียบกับตัวอย่างในบทที่ 29 — ใช้เป็นแนวทางในแล็บของกลุ่ม

ตำรา / แนวทางเดิม	แนวปฏิบัติปัจจุบัน
connection: local + host/username/password ในทุก task	<code>ansible_connection: network_cli</code> + ตัวแปรใน inventory + ansible-vault
เรียกโมดูลด้วยชื่อสั้น <code>ios_config</code>	<code>cisco.ios.ios_config</code> ชื่อเต็ม (FQCN) จาก Collections ตั้งแต่ Ansible 2.10
ส่งบรรทัด CLI ตรง ๆ	Resource modules (state: merged / replaced / overridden) idempotent จริง
EEM 1.0: event จาก syslog / SNMP เท่านั้น	EEM 4.0 SMTP ผ่าน TLS, กำหนด CPU/memory threshold, IPv6 routing detector, สถิติการรัน
ตอบสนอง event ได้เฉพาะในเครื่องเดียว (EEM)	Event-Driven Ansible (rulebook) รับ event ทั้งเครือข่าย · Salt reactor
คุยกับอุปกรณ์ผ่าน CLI แล้ว parse ข้อความ	Model-driven: NETCONF / RESTCONF / gNMI + YANG, streaming telemetry
Agent บนอุปกรณ์เครือข่าย (Puppet / Chef)	ลดบทบาท → งานเครือข่ายหันไปใช้ agentless และ API ของ controller

❓ **แก้ไขผิดพลาดในสไลด์ต้นฉบับ:** คำสั่งที่ถูกคือ `ansible-doc` (ไม่มี s) · YAML ย่อมาจาก "YAML Ain't Markup Language"

Lab Topology และสภาพแวดล้อม

Cisco Modeling Labs (DevNet Sandbox) · Cisco IOSv 15.9 × 2 · Ubuntu 24.04 เป็น Ansible control node และ syslog server



อุปกรณ์	Interface	IP Address
Host	mgmt0	192.168.100.10/24
R1	Gi0/0	192.168.100.11/24
	Gi0/1	10.12.12.1/30
	Lo0	1.1.1.1/32
R2	Gi0/0	192.168.100.12/24
	Gi0/1	10.12.12.2/30
	Lo0	2.2.2.2/32

ซอฟต์แวร์บน Automation Host

- ansible-core 2.21 + cisco.ios, ansible.netcommon
- paramiko (SSH library ของ network_cli)
- rsyslog รับ log · Packet Capture ของ CML
- **ไม่ใช่ Packet Tracer:** ไม่รองรับ EEM และต่อ Ansible ไม่ได้

Step 0 — เตรียมอุปกรณ์ (Day-0)

คอนฟิกขั้นต้นบน R1 และ R2 (ในแล็บใส่ไว้ในไฟล์ lab ของ CML ให้บูตพร้อมใช้) — หลังจากนั้นทุกอย่างทำผ่าน Ansible

```

R1 — Day-0 config (R2: เปลี่ยน hostname และ IP)

hostname R1
ip domain name lab.local
username admin privilege 15 secret Cisco@123
crypto key generate rsa modulus 2048
ip ssh version 2
!
interface GigabitEthernet0/0
 description MGMT
 ip address 192.168.100.11 255.255.255.0
 no shutdown
!
line vty 0 4
 login local
 transport input ssh
!
archive
 log config
 logging enable
 notify syslog contenttype plaintext
!
logging host 192.168.100.10
    
```

รหัสผ่านในสไลด์ใช้ในแล็บเท่านั้น

domain + crypto key	สร้าง RSA key 2048 bit — ถ้าไม่มี key บริการ SSH จะไม่ทำงาน
ip ssh version 2	บังคับใช้ SSHv2 และปิด SSHv1 ที่ไม่ปลอดภัย
privilege 15	Ansible login แล้วเข้า privileged EXEC ทันที ไม่ต้องใช้ enable
transport input ssh	VTY รับเฉพาะ SSH (ปิด Telnet ที่ส่งรหัสผ่านเป็นข้อความธรรมดา)
archive log config	บันทึกทุกคำสั่งที่ถูกส่งเข้ามาและส่งออกเป็น syslog → ใช้ audit งานของ Ansible
logging host	ส่ง syslog ไปที่ automation host ใช้ทั้ง audit และเป็นแหล่ง event



ตรวจสอบความพร้อมก่อนรัน Ansible

show ip ssh · show crypto key mypubkey rsa

show run | section line vty

Demo 1 — Inventory และ Playbook

inventory.yml — desired state ชุดเดียวใช้ได้ทุก router

```
all:
  children:
    routers:
      hosts:
        R1: {ansible_host: 192.168.100.11,
              rid: 1.1.1.1, p2p_ip: 10.12.12.1}
        R2: {ansible_host: 192.168.100.12,
              rid: 2.2.2.2, p2p_ip: 10.12.12.2}
      vars:
        ansible_connection: ansible.netcommon.network_cli
        ansible_network_os: cisco.ios.ios
        ansible_user: admin
        ansible_password: "{{ vault_admin_pw }}"
```

จุดสังเกต

- ตัวแปรต่อเครื่อง (rid, p2p_ip) อยู่ใน inventory → playbook เดียวใช้ได้ทุก router
- รหัสผ่านอยู่ใน group_vars/routers/vault.yml ที่เข้ารหัสด้วย ansible-vault (AES256)
- `ios_config` + "no shutdown" จะ changed ทุกรอบ เพราะ running-config ไม่แสดงบรรทัดนี้ → ใช้ `ios_interfaces`

site.yml

```
- name: Deploy OSPF area 0
  hosts: routers
  gather_facts: false
  tasks:
    - name: L3 interfaces (resource module)
      cisco.ios.ios_l3_interfaces:
        state: merged
        config:
          - {name: Loopback0, ipv4: [{address: "{{ rid }}/32"}]}
          - {name: GigabitEthernet0/1,
              ipv4: [{address: "{{ p2p_ip }}/30"}]}
    - name: Enable Gi0/1 (idempotent no shutdown)
      cisco.ios.ios_interfaces:
        state: merged
        config: [{name: GigabitEthernet0/1, enabled: true}]
    - name: OSPF process 1
      cisco.ios.ios_config:
        parents: router ospf 1
        lines:
          - router-id {{ rid }}
          - network 10.12.12.0 0.0.0.3 area 0
          - network {{ rid }} 0.0.0.0 area 0
    - name: Verify adjacency is FULL
      cisco.ios.ios_command:
        commands: show ip ospf neighbor
        wait_for: result[0] contains FULL
        retries: 12
        interval: 5
        when: not ansible_check_mode
```

Demo 1 — Dry-run → Apply → Idempotency

รัน playbook 3 รอบ: ตรวจสอบด้วย --check, ใช้งานจริง แล้วรันซ้ำเพื่อพิสูจน์ว่าไม่มีการเปลี่ยนแปลงซ้ำ

automation-host (Ubuntu) — bash

```
$ ansible-inventory -i inventory.yml --graph
```

```
@all:
  |--@ungrouped:
  |--@routers:
    |--R1
    |--R2
```

```
$ ansible-playbook -i inventory.yml site.yml -J --check -v
```

```
TASK [OSPF process 1] *****
changed: [R1] => {"banners": {}, "changed": true, "commands": ["router ospf 1",
  "router-id 1.1.1.1", "network 10.12.12.0 0.0.0.3 area 0", ...]}
TASK [Verify adjacency is FULL] *****
skipping: [R1] => {"false_condition": "not ansible_check_mode", ...}
```

```
$ ansible-playbook -i inventory.yml site.yml -J
```

```
PLAY RECAP *****
R1  : ok=4    changed=3    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
R2  : ok=4    changed=3    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

```
$ ansible-playbook -i inventory.yml site.yml -J
```

```
PLAY RECAP *****
R1  : ok=4    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
R2  : ok=4    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Dry-run

--check

แสดงคำสั่งที่ “จะ” ส่ง โดยไม่แตะอุปกรณ์



รอบที่ 1

changed = 3

ส่งเฉพาะคำสั่งที่ยังไม่มีใน running-config



รอบที่ 2

changed = 0

สถานะตรง desired state แล้ว ไม่ส่งอะไรเลย

ผลลัพธ์จริงจากแล็บ Cisco CML (DevNet Sandbox) — ย่อช่องว่างบางส่วนให้พอดีสไลด์

Demo 1 — ตรวจสอบผลงานอุปกรณ์

ยืนยันทั้ง control plane (OSPF) และ audit trail ว่าคำสั่งมาจาก Ansible จริง

```

R1 — verification

R1# show ip ospf neighbor
Neighbor ID      Pri   State           Dead Time   Address        Interface
2.2.2.2          1     FULL/DR         00:00:31    10.12.12.2     GigabitEthernet0/1

R1# show ip route ospf | begin Gateway
Gateway of last resort is not set
  2.0.0.0/32 is subnetted, 1 subnets
O       2.2.2.2 [110/2] via 10.12.12.2, 00:00:40, GigabitEthernet0/1

R1# show archive log config all | include router-id
40      0          admin@vty1      | router-id 1.1.1.1

R1# show logging | include CONFIG_I
*Sep 16 23:20:38.643: %SYS-5-CONFIG_I: Configured from console
by admin on vty1 (192.168.100.10)
    
```

ผลลัพธ์จริงจากแล็บ Cisco CML (DevNet Sandbox) — ย่อช่องว่างบางส่วนให้พอดีสไลด์

สิ่งที่ยืนยันได้

- ✓ FULL/DR adjacency สมบูรณ์ แลก LSA ครบ
- ✓ o [110/2] AD 110, cost 1 + 1 ผ่าน Gi0/1
- ✓ admin@vty1 ทุกคำสั่งถูกส่งผ่าน SSH session ของ Ansible
- ✓ (192.168.100.10) syslog ระบุ IP ต้นทางคือ control node

Packet capture บนลิงก์ MGMT

- **Filter:** tcp.port == 22
- TCP handshake .10 → .11:22
- SSH-2.0 banner + Key Exchange
- payload ถูกเข้ารหัสทั้งหมด (config ไม่รู้)
- ไม่มี port อื่นของ agent = agentless จริง

Demo 2 — EEM เก็บหลักฐานเมื่อ OSPF ล่ม

ดั่งบน R1: เมื่อ syslog ตรง pattern ให้เก็บ neighbor ทันที รอ SPF 6 วินาทีแล้วเก็บ routing table และส่ง syslog critical

● ● ● R1 — applet configuration

```
R1(config)# event manager applet OSPF-NBR-DOWN
event syslog pattern "OSPF-5-ADJCHG.*FULL to DOWN" ratelimit 60 maxrun 60
action 1.0 cli command "enable"
action 1.5 cli command "delete /force flash0:ospf-ev.txt"
action 2.0 cli command "show ip ospf neighbor | redirect flash0:ospf-ev.txt"
action 2.5 wait 6
action 3.0 cli command "show ip route ospf | append flash0:ospf-ev.txt"
action 4.0 syslog priority critical msg "OSPF nbr lost - evidence saved"
```

● ● ● R1 — ตรวจสอบการลงทะเบียน

```
R1# show event manager policy registered
No.  Class  Type  Event Type  Trap  Name
1    applet  user  syslog      Off   OSPF-NBR-DOWN
pattern {OSPF-5-ADJCHG.*FULL to DOWN}
maxrun 60.000
ratelimit 60.000
action 1.0 ... action 4.0
R1# debug event manager action cli
```

● ● ● R1 — เมื่อเกิดเหตุ (debug + history)

```
%OSPF-5-ADJCHG: Process 1, Nbr 2.2.2.2 on GigabitEthernet0/1 from
FULL to DOWN, Neighbor Down: Dead timer expired
%HA_EM-6-LOG: OSPF-NBR-DOWN : ... : IN : R1#delete /force flash0:ospf-ev.txt
%HA_EM-6-LOG: OSPF-NBR-DOWN : ... : IN : R1#show ip ospf neighbor | redirect ...
%HA_EM-6-LOG: OSPF-NBR-DOWN : ... : IN : R1#show ip route ospf | append ...
%HA_EM-2-LOG: OSPF-NBR-DOWN: OSPF nbr lost - evidence saved
R1# show event manager history events
No.  Job Id  Proc Status  Time of Event  Type  Name
2    2      Actv success  Wed Sep16 23:28:25 2026  syslog  applet: OSPF-NBR-DOWN
```

ผลลัพธ์จริงจากแล็บ Cisco CML (DevNet Sandbox) — ย่อช่องว่างบางส่วนให้พอดีสไลด์

Troubleshooting #1 — Misconfiguration

สถานการณ์: RSA key บน R2 ถูกลบ (เช่น เปลี่ยนเครื่องแล้วทำ Day-0 ไม่ครบ) → Ansible เชื่อมต่อ R2 ไม่ได้

- 1 **อาการ (Symptom)**
playbook ล้มเหลวเฉพาะ R2: "Unable to connect to port 22"
- 2 **Layer 3 ถึงกันไหม?**
ping 192.168.100.12 → ผ่าน (ตัดปัญหา routing / สาย)
- 3 **Layer 4 port เปิดไหม?**
nc -zv → Connection refused → ปัญหาอยู่ที่บริการบนอุปกรณ์
- 4 **ตรวจบน R2**
show ip ssh → SSH Disabled · ไม่มี RSA key pair
- 5 **แก้ไขและยืนยัน**
สร้าง key → nc สำเร็จ → ssh-keygen -R (host key เปลี่ยน) → รัน playbook → failed=0

```

diagnose → fix → verify

$ ansible-playbook site.yml -J
fatal: [R2]: FAILED! => {"changed": false,
  "msg": "[Errno None] Unable to connect to port 22 on 192.168.100.12"}
$ ping -c 2 192.168.100.12
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
$ nc -zv 192.168.100.12 22
nc: connect to 192.168.100.12 port 22 (tcp) failed: Connection refused
R2# show ip ssh
SSH Disabled - version 2.0
%Please create RSA keys to enable SSH (and of atleast 768 bits for SSH v2).
R2(config)# crypto key generate rsa modulus 2048
% Generating 2048 bit RSA keys, keys will be non-exportable...
[OK] (elapsed time was 0 seconds)
%SSH-5-ENABLED: SSH 2.0 has been enabled
$ nc -zv 192.168.100.12 22
Connection to 192.168.100.12 22 port [tcp/ssh] succeeded!
$ ssh-keygen -R 192.168.100.12
# Host 192.168.100.12 found: line 2
/home/grp9/.ssh/known_hosts updated.
$ ansible-playbook site.yml -J
R2 : ok=4  changed=0  unreachable=0  failed=0  skipped=0  rescued=0
    
```

ผลลัพธ์จริงจากแล็บ Cisco CML (DevNet Sandbox) — ย่อช่องว่างบางส่วนให้พอดีสไลด์

Troubleshooting #2 — Network Failure

จำลองลิงก์ R1-R2 ล่ม แล้วติดตามการเปลี่ยนแปลงของ control plane จนกู้คืนได้



```

●●● วินิจฉัย (R1 และ R2)

R1# show ip ospf neighbor
R1#                                     (no neighbors)
R1# show ip route 2.2.2.2
% Network not in table
R1# show ip ospf interface brief
Interface  PID  Area  IP Address/Mask  Cost  State  Nbrs  F/C
Lo0        1    0     1.1.1.1/32       1     LOOP  0/0
Gi0/1      1    0     10.12.12.1/30    1     DR    0/0
R2# show ip interface brief | include 0/1
GigabitEthernet0/1  10.12.12.2  YES manual administratively down down
    
```

```

●●● แก้ไขบน R2 → ยืนยันบน R1

R2 (config-if) # no shutdown
R1: %OSPF-5-ADJCHG: Process 1, Nbr 2.2.2.2 on
      GigabitEthernet0/1 from LOADING to FULL, Loading Done
R1# show ip route ospf | include 2.2.2.2
O        2.2.2.2 [110/2] via 10.12.12.2, 00:00:05, GigabitEthernet0/1
R1# ping 2.2.2.2 source loopback0
!!!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 1/1/2 ms
    
```

ผลลัพธ์จริงจากแล็บ Cisco CML (DevNet Sandbox) — ย่อช่องว่างบางส่วนให้พอดีสไลด์

Troubleshooting Toolkit

อาการที่พบบ่อย → ขั้นที่ควรสงสัย → คำสั่งวินิจฉัย → สิ่งที่ต้องดูและวิธีแก้

อาการ	ขั้นที่สงสัย	คำสั่งวินิจฉัย	สิ่งที่ต้องดู / วิธีแก้
Ansible: “Unable to connect to port 22”	SSH service	<code>nc -zv <ip> 22</code> <code>show ip ssh</code>	Connection refused → สร้าง RSA key, ip ssh version 2
Ansible: “Authentication failed”	Credential / VTY	<code>ansible-inventory --host R2</code> <code>show run section line vty</code>	user/password ใน vault, login local, transport input ssh
Ansible: “host key mismatch”	SSH host key	<code>ssh-keygen -R 192.168.100.12</code>	router สร้าง RSA key ใหม่ → ลบ key เก่าที่ control node จำไว้
task เป็น changed ทุกรอบ	Playbook logic	<code>ansible-playbook site.yml --check -v</code>	ดู commands ที่ถูกส่งเข้า → เปลี่ยนเป็น resource module
EEM applet ไม่ทำงาน	Event (regex)	<code>show event manager policy registered</code> <code>show logging include ADJCHG</code>	ยังไม่ลงทะเบียน applet หรือ pattern ไม่ตรงกับข้อความจริง
EEM ขึ้น abort (เจอจริงตอนซ้อม)	Action	<code>debug event manager action cli</code> <code>show users</code>	คำสั่งที่ redirect เพราะ IOS ตามเขียนทับไฟล์ → delete /force ก่อน, เพิ่ม maxrun
OSPF neighbor ไม่ขึ้น	Control plane	<code>show ip ospf interface brief</code> <code>debug ip ospf adj</code>	area, hello/dead timer, network type, authentication ไม่ตรงกัน

Head-to-Head Comparison

เปรียบเทียบเครื่องมือในบทที่ 29 กับทางเลือกแบบเขียนโปรแกรมเอง (Python: Netmiko / Nornir)

ปัจจัย	EEM (ใช้ในเดโม)	Ansible (ใช้ในเดโม)	Puppet	Chef	SaltStack	Python (Netmiko / Nornir)
สถาปัตยกรรม	On-box ในอุปกรณ์	Control node → hosts	Server + agent	Infra Server + client	Master + minion	สคริปต์บนเครื่องผู้ดูแล
ต้องมี agent	ไม่ต้อง	ไม่ต้อง	ต้อง (Bolt: ไม่ต้อง)	ต้อง	ต้อง (Salt SSH: ไม่ต้อง)	ไม่ต้อง
รูปแบบการทำงาน	Event-driven	Push (+ Event-Driven Ansible)	Pull ทุก 30 นาที	Pull ทุก 30 นาที	Push + event bus	Push (imperative)
ภาษา	CLI applet / Tcl	YAML + Jinja2	Puppet DSL	Ruby DSL	YAML + Jinja2	Python
Transport / Port	ภายในเครื่อง	SSH 22 / HTTPS API	HTTPS 8140	HTTPS 443	ZeroMQ 4505/4506	SSH 22
Idempotent	ต้องออกแบบเอง	ใช่ (modules)	ใช่	ใช่	ใช่	ต้องเขียนเอง
จุดแข็ง	ตอบสนองทันที แม้ไม่มี server	เรียนง่าย รองรับหลาย vendor	บังคับ compliance ขนาดใหญ่	ยืดหยุ่นด้วย Ruby	เร็ว รองรับ event-driven	ยืดหยุ่นสูงสุด

คอลัมน์พื้นที่ทอง = เครื่องมือที่ใช้ในการสาธิต · Puppet runinterval และ chef-client daemon interval มีค่าเริ่มต้น 30 นาที (1800 วินาที)

Trade-Off Matrix

คะแนน 1–5 (ยิ่งมากยิ่งดีในมิตินั้น) — การประเมินเชิงคุณภาพของกลุ่ม สำหรับงานเครือข่ายองค์กร

เครื่องมือ	Scalability	Security	ความง่าย (Complexity ต่ำ)	Vendor Support (อุปกรณ์เครือข่าย)	ต้นทุนต่ำ (Cost)
EEM					
Ansible					
Puppet					
Chef					
SaltStack					
Python (Nornir)					

จุด 1 จุด = 1 คะแนน · สีเข้ม = จุดแข็งของเครื่องมือ

เหตุผลประกอบ

Scalability: Puppet/Chef/Salt ออกแบบเพื่อ node หลายหมื่นตัว EEM จำกัดที่ละเครื่อง

Security: agentless ไม่เปิด port เพิ่ม; Salt master เคยมีช่องโหว่ร้ายแรง (CVE-2020-11651)

ความง่าย: YAML อ่านง่ายกว่า DSL, Ruby หรือ Python

Vendor: Ansible มี collection ของ Cisco, Juniper, Arista ฯลฯ; EEM ใช้ได้เฉพาะ Cisco

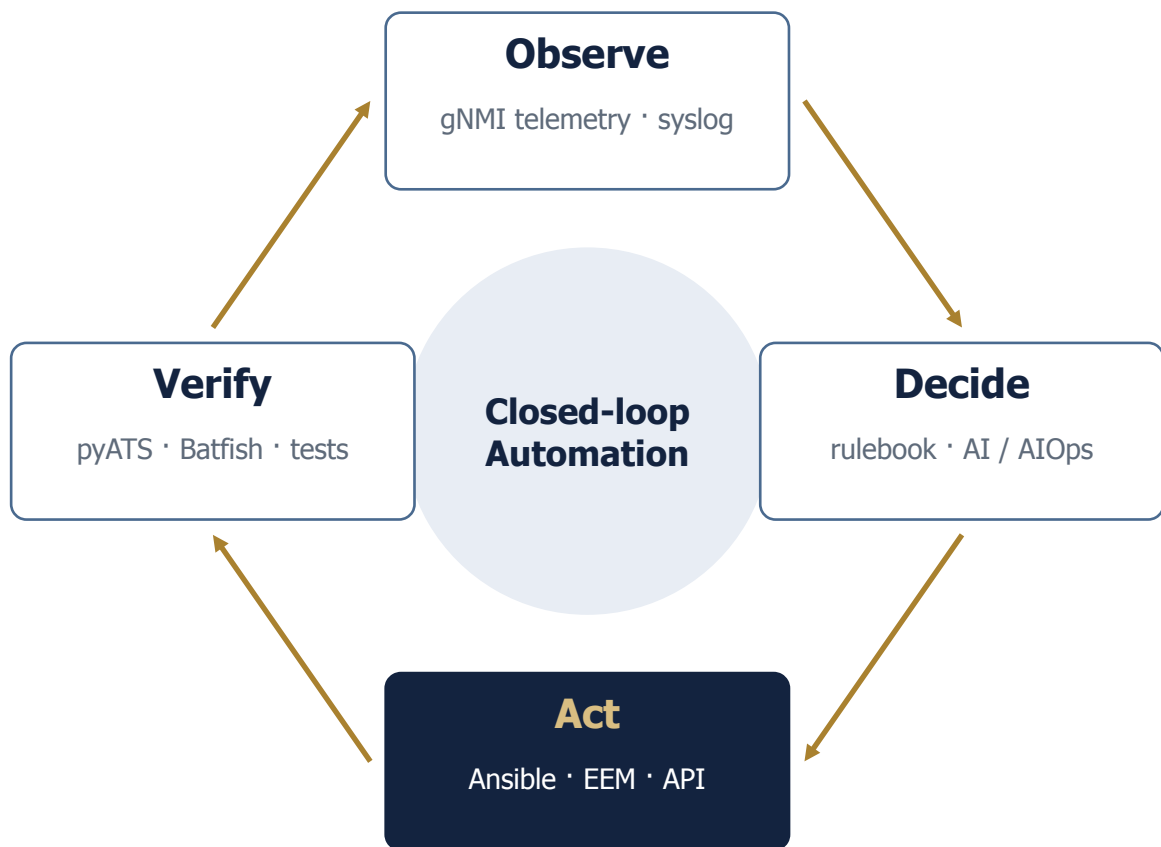
ต้นทุน: EEM มากับ IOS; Puppet Core / Chef มี license เชิงพาณิชย์

เลือกใช้เมื่อไร

- EEM แก้ไข / เก็บหลักฐานทันทีบนเครื่อง
- Ansible desired state ทั้งเครือข่าย
- Puppet / Chef / Salt fleet ของ server
- Python (Nornir) logic เฉพาะทางที่ซับซ้อน

Next-Gen Evolution (3–5 ปีข้างหน้า)

จาก “สั่งงานตามคำสั่ง” ไปสู่ “ระบบที่สังเกต ตัดสินใจ ลงมือ และตรวจสอบเองแบบวงปิด”



Event-driven & closed-loop

Event-Driven Ansible / Salt reactor ต่อยอดแนวคิด EEM ไปทั้งเครือข่าย: event → rulebook → แก้ไขอัตโนมัติ



Model-driven programmability

NETCONF / RESTCONF / gNMI + YANG และ streaming telemetry แทนการ parse ข้อความ CLI



AI-assisted operations

สร้าง playbook ด้วย GenAI (เช่น Ansible Lightspeed), AIOps วิเคราะห์ log, AI agent ที่มีคนอนุมัติก่อนลงมือ



NetDevOps / GitOps

Source of Truth (NetBox) + Git + CI/CD ทดสอบก่อนใช้งานจริงบน digital twin



Zero Trust & Cloud

บังคับ policy และ least privilege อัตโนมัติ, เก็บ secret ใน vault, จัดการ cloud network ด้วย Terraform / API

Industry Takeaway

ทำไมวิศวกรเครือข่ายยุคใหม่ยังต้องเข้าใจทั้ง automation และพื้นฐานของ protocol



Automation ขยายผลได้ทั้ง สิ่งที่ถูกและสิ่งที่ผิด

config ผิดเพียงบรรทัดเดียวถูกส่งไปหลายร้อยเครื่องได้ในไม่กี่วินาที ผู้เขียน automation จึงต้องเข้าใจ protocol และ CLI ที่อยู่เบื้องหลังจริง

สัญญาณจากอุตสาหกรรม: Cisco เปลี่ยน DevNet เป็น CCNA / CCNP / CCIE Automation (3 ก.พ. 2026)



Troubleshooting ยังใช้ CLI

TS #2 อธิบายได้เพราะเข้าใจ Dead timer, LSA และ SPF — เครื่องมือไม่ได้บอกเหตุผลให้



ต้องรู้ผลลัพธ์ที่ถูกต้อง

การเขียน wait_for หรือ test ต้องรู้ output ที่คาดหวัง เช่น FULL และ [110/2]



ทักษะใหม่ที่ต้องมี

YAML / Jinja2, Git, Python, REST API และ YANG data model



เป็นทักษะหลักของสายงาน

automation อยู่ใน CCNP Enterprise (ENCOR) และเป็นสาย CCNP Automation โดยตรง

สรุป

Key Takeaways

1

EEM = automation บนอุปกรณ์

event → action ทันที ไม่ต้องพึ่ง server — เหมาะกับ self-healing และเก็บหลักฐาน

2

เลือก agent-based หรือ agentless ตามปลายทาง

Puppet / Chef / Salt เหมาะกับ fleet ของ server; Ansible เหมาะกับ router / switch

3

ใช้ร่วมกันได้ และยังต้องเข้าใจ control plane

Ansible กำหนด desired state + EEM เผื่อระวัง — แต่การวิเคราะห์ปัญหายังต้องใช้ความรู้ protocol



Q & A

ขอบคุณ

คำถามและข้อเสนอแนะ

Cheat sheet: [grp9-Automation-CheatSheet.pdf](#)

คำถามที่คาดว่าจะถูกถาม

Q1 ทำไม R1 รู้ว่า neighbor ล่มช้ากว่า R2?

R2 เห็น interface down จึงล้าง adjacency ทันที แต่ลิงก์ฝั่ง R1 ยัง up จึงต้องรอ Dead timer 40 วินาที ($4 \times \text{Hello}$) — ลดเวลาได้ด้วย BFD (RFC 5880)

Q3 ทำไม applet มี “enable” และ “delete /force”?

session ของ EEM เริ่มที่ user EXEC จึงต้อง enable ก่อน และ redirect จะถามยืนยันถ้ามีไฟล์เดิม ทำให้ applet ค้างจน abort (เจอจริงตอนซ้อม)

Q5 ทำไมไม่ใช้ ios_config สั่ง no shutdown?

running-config ไม่แสดงบรรทัด “no shutdown” โมดูลจึงเห็นว่าต่างทุกครั้ง → changed ทุกรอบ (ไม่ idempotent)

Q2 Control plane เปลี่ยนอย่างไรเมื่อถูกลิงก์?

Down → Init → 2-Way → ExStart → Exchange → Loading → Full
แลก LSDB, รัน SPF แล้วติดตั้ง O 2.2.2.2 ลง RIB/FIB

Q4 ทำไมต้อง wait 6 ก่อนเก็บ routing table?

IOS หน่วง SPF ~5 วินาที (SPF throttle) ก่อนถอน route ถ้าเก็บทันทียังเห็น O 2.2.2.2 อยู่ (เจอจริงตอนซ้อม) · ratelimit 60 กันทำงานรวตอน flap

Q6 EEM กับ Ansible ใช้ร่วมกันอย่างไร?

EEM ตอบสนองทันทีในเครื่องและส่ง syslog → Event-Driven Ansible รับ event ไปแก้ไขหรือเปิด ticket ระดับเครือข่าย

แหล่งอ้างอิง



- [1] Cisco Networking Academy. CCNP Enterprise: Core Networking — Chapter 29: Introduction to Automation Tools (Instructor Materials).
- [2] Edgeworth, B., Garza Rios, R., Gooley, J., & Hucaby, D. CCNP and CCIE Enterprise Core ENCOR 350-401 Official Cert Guide. Cisco Press.
- [3] Cisco. System Management Configuration Guide, Cisco IOS XE 17.x — Embedded Event Manager Overview; Writing EEM Policies Using the Cisco IOS CLI.
- [4] Cisco. Troubleshoot and Test EEM Scripts (Document ID 217013) · EEM 4.0 Data Sheet · DevNet Sandbox: Cisco Modeling Labs.
- [5] Ansible Community Documentation — Network Automation (network_cli), cisco.ios collection · endoflife.date/ansible-core.
- [6] IETF RFC 2328 (OSPFv2), RFC 4251–4254 (SSH), RFC 5424 (Syslog), RFC 5880 (BFD), RFC 6241 (NETCONF), RFC 7950 (YANG 1.1), RFC 8040 (RESTCONF).
- [7] Vox Pupuli — OpenVox Project · Salt Project — Salt 3008 release notes · Progress — Introducing Chef Infra Client 19.
- [8] Gartner (18 Sep 2024). Gartner Says 30% of Enterprises Will Automate More Than Half of Their Network Activities by 2026.
- [9] Cisco Blogs (Learning & Certifications). CCNP Automation: A Renamed Certification, Reimagined.